

CodeMesh 2013, December 4
dean.wampler@typesafe.com
[@deanwampler](https://twitter.com/deanwampler)
polyglotprogramming.com/talks

What's Ahead for Big Data?



Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Copyright © Dean Wampler, 2011–2013, All Rights Reserved. Photos can only be used with permission. Otherwise, the content is free to use.

Photo: John Hancock Center, Michigan Ave. Chicago, Illinois USA

A photograph of the Chicago skyline, featuring several prominent skyscrapers like the Willis Tower and the Chicago Board of Trade Building. The Chicago River flows in the foreground, with a bridge and a small boat visible. The text 'Consultant at Typesafe' is overlaid in a large, white, sans-serif font.

Consultant at Typesafe

Dean Wampler...

Copyright © 2011-2013 Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Typesafe builds tools for creating Reactive Applications, <http://typesafe.com/platform>. See also the Reactive Manifesto, <http://www.reactivemanifesto.org/>

Photo: The Chicago River

A photograph of the Chicago skyline, featuring several prominent skyscrapers like the Willis Tower and the Chicago Board of Trade Building. The city is viewed from across a body of water, likely Lake Michigan, with a bridge visible in the foreground. The text is overlaid on the image in a large, white, sans-serif font.

Founder,
Chicago-Area Scala
Enthusiasts
and co-organizer,
Chicago Hadoop User Group

Dean Wampler...

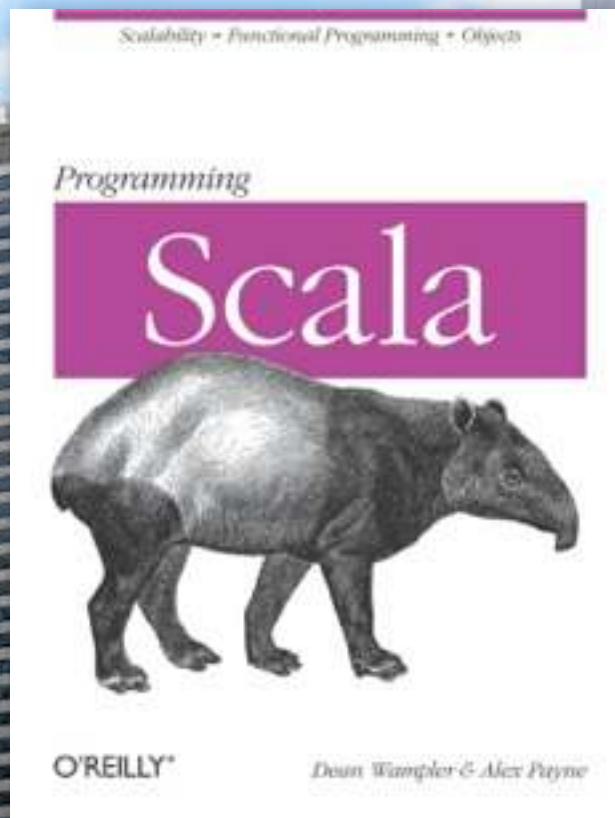
Copyright © 2011-2013 Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

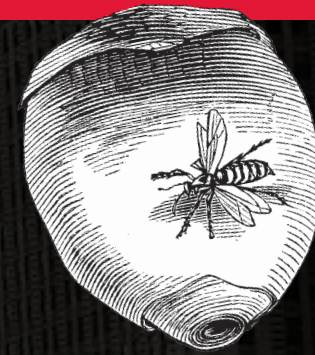
I've been doing Scala for 6 years and Big Data for 3.5 years.



Functional
Programming



Hive



Dean Wampler...

Copyright © 2011-2013 Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

My books...

What Is Big Data?



DevOps Borat @DEVOPS_BORAT

8 Jan

Big Data is any thing which is crash Excel.

Expand



DevOps Borat @DEVOPS_BORAT

6 Feb

Small Data is when is fit in RAM. Big Data is when is crash because is not fit in RAM.

Expand

Big Data

Data so big that traditional solutions are too slow, too small, or too expensive to use.



Hat tip: Bob Korbus

It's a buzz word, but generally associated with the problem of data sets too big to manage with traditional SQL databases. A parallel development has been the NoSQL movement that is good at handling semistructured data, scaling, etc.

3 Trends

Copyright © 2011-2013, Dean Wampler. All Rights Reserved

Wednesday, December 4, 13

Three prevailing trends driving data-centric computing.

Photo: Prizker Pavilion, Millenium Park, Chicago (designed by Frank Gehry)

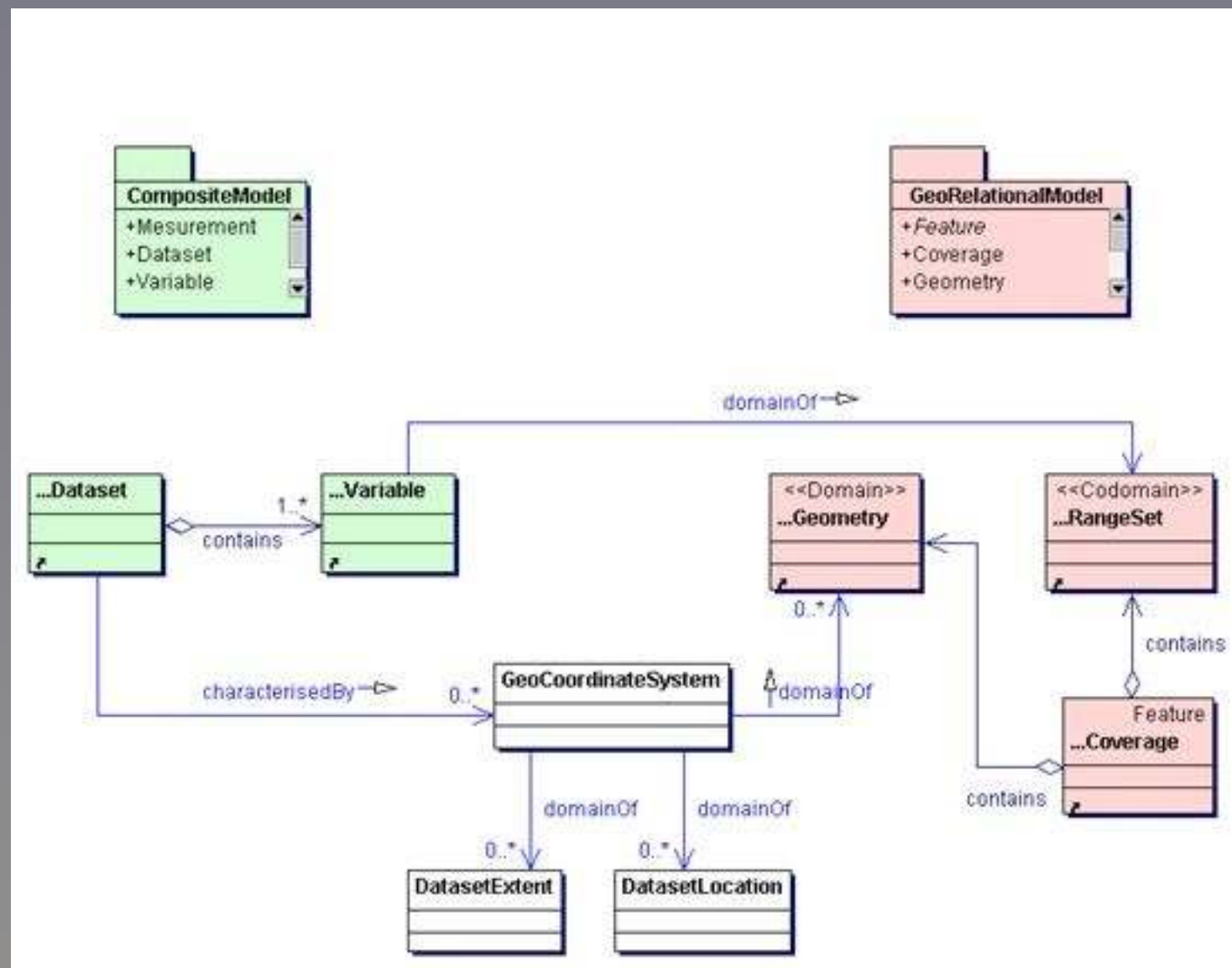
Data Size ↑



Wednesday, December 4, 13

Data volumes are obviously growing... rapidly.
Facebook now has over 600PB (Petabytes) of data in Hadoop clusters!

Formal Schemas ↓



There is less emphasis on “formal” schemas and domain models, i.e., both relational models of data and OO models, because data schemas and sources change rapidly, and we need to integrate so many disparate sources of data. So, using relatively-agnostic software, e.g., collections of things where the software is more agnostic about the structure of the data and the domain, tends to be faster to develop, test, and deploy. Put another way, we find it more useful to build somewhat agnostic applications and drive their behavior through data...

Data-Driven Programs ↑



Wednesday, December 4, 13

This is the 2nd generation “Stanley”, the most successful self-driving car ever built (by a Google–Stanford) team. Machine learning is growing in importance. Here, generic algorithms and data structures are trained to represent the “world” using data, rather than encoding a model of the world in the software itself. It’s another example of generic algorithms that produce the desired behavior by being application agnostic and data driven, rather than hard-coding a model of the world. (In practice, however, a balance is struck between completely agnostic apps and some engineering towards for the specific problem, as you might expect...)

Probabilistic Models vs. Formal Grammars

[tor.com/blogs/...](http://www.tor.com/blogs/...)

Norvig vs. Chomsky and the Fight for the Future of AI

KEVIN GOLD

When the Director of Research for Google compares one of the most highly regarded linguists of all time to Bill O'Reilly, you know it is *on*. Recently, Peter Norvig, Google's Director of Research and co-author of [the most popular artificial intelligence textbook in the world](#), wrote a [webpage](#) extensively criticizing Noam Chomsky, arguably the most influential linguist in the world. Their disagreement points to a revolution in artificial intelligence that, like many revolutions, threatens to destroy as much as it improves. Chomsky, one of the old guard, wishes for an elegant theory of intelligence and language that looks past human fallibility to try to see simple structure underneath. Norvig, meanwhile, represents the new philosophy: truth by statistics,



Chomsky photo by Duncan Rawlinson and his Online Photography School. Norvig photo by Peter Norvig

||

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

An interesting manifestation of this trend is the public argument between Noam Chomsky and Peter Norvig on the nature of language. Chomsky long ago proposed a hierarchical model of formal language grammars. Peter Norvig is a proponent of probabilistic models of language. Indeed all successful automated language processing systems are probabilistic.

<http://www.tor.com/blogs/2011/06/norvig-vs-chomsky-and-the-fight-for-the-future-of-ai>

Big Data Architectures

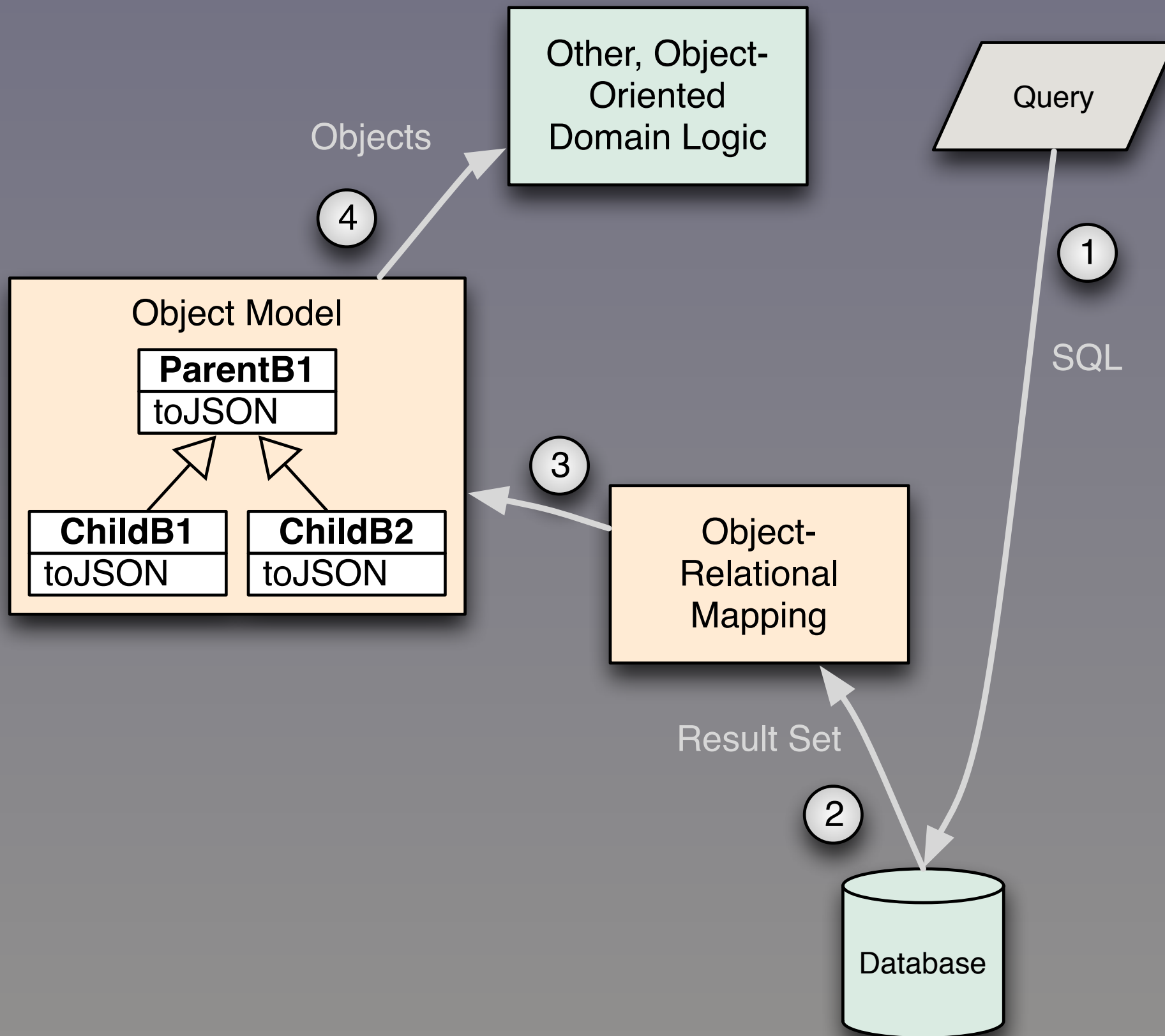


Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

What should software architectures look like for these kinds of systems?

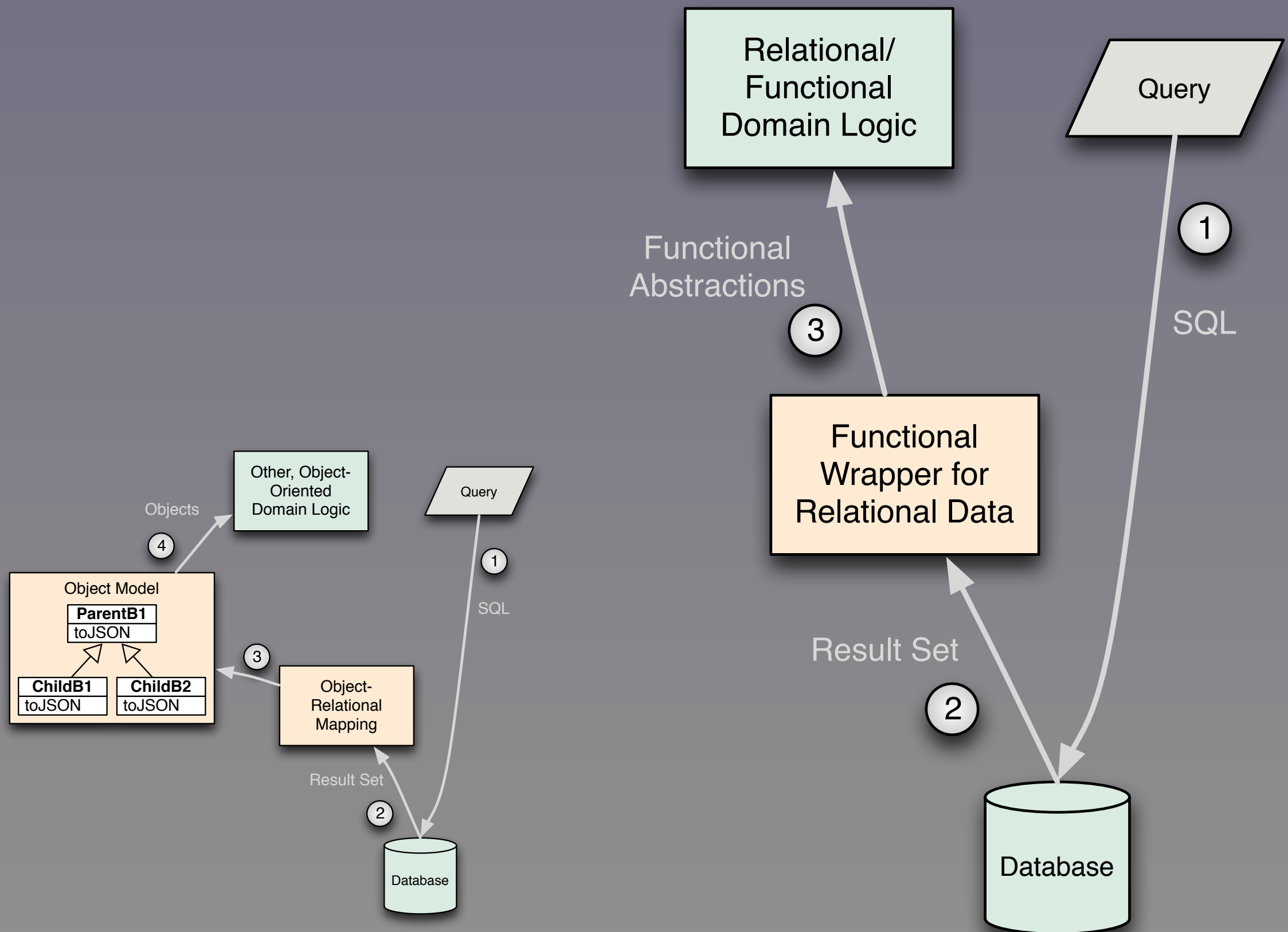
Photo: Cloud Gate (a.k.a. “The Bean”) in Millenium Park, Chicago, on a cloudy day.



Wednesday, December 4, 13

Traditionally, we've kept a rich, in-memory domain model requiring an ORM to convert persistent data into the model. This is resource overhead and complexity we can't afford in big data systems. Rather, we should treat the result set as it is, a particular kind of collection, do the minimal transformation required to exploit our collections libraries and classes representing some domain concepts (e.g., Address, StockOption, etc.), then write functional code to implement business logic (or drive emergent behavior with machine learning algorithms...)

The toJSON methods are there because we often convert these object graphs back into fundamental structures, such as the maps and arrays of JSON so we can send them to the browser!



- Focus on:

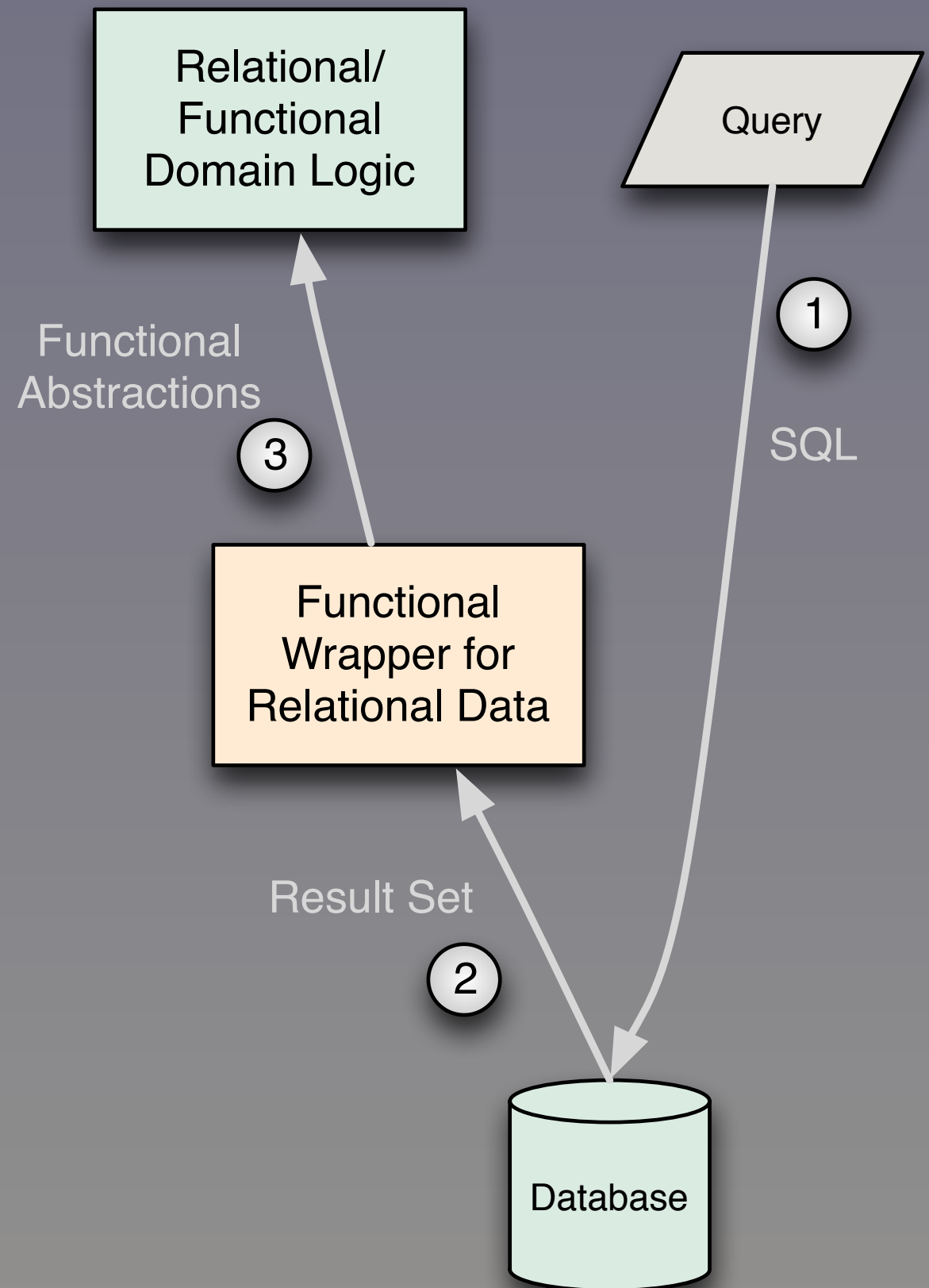
- Lists

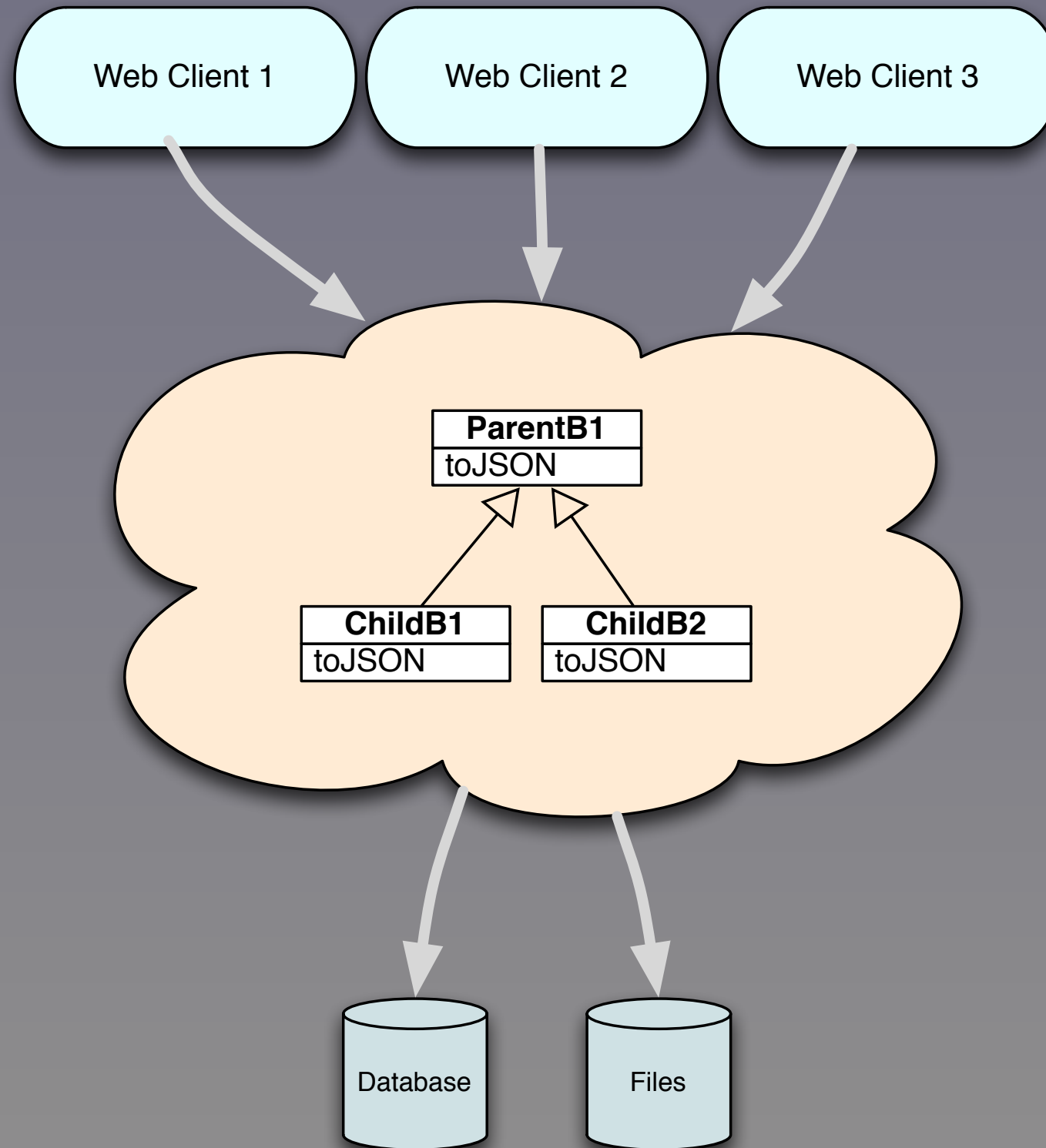
- Maps

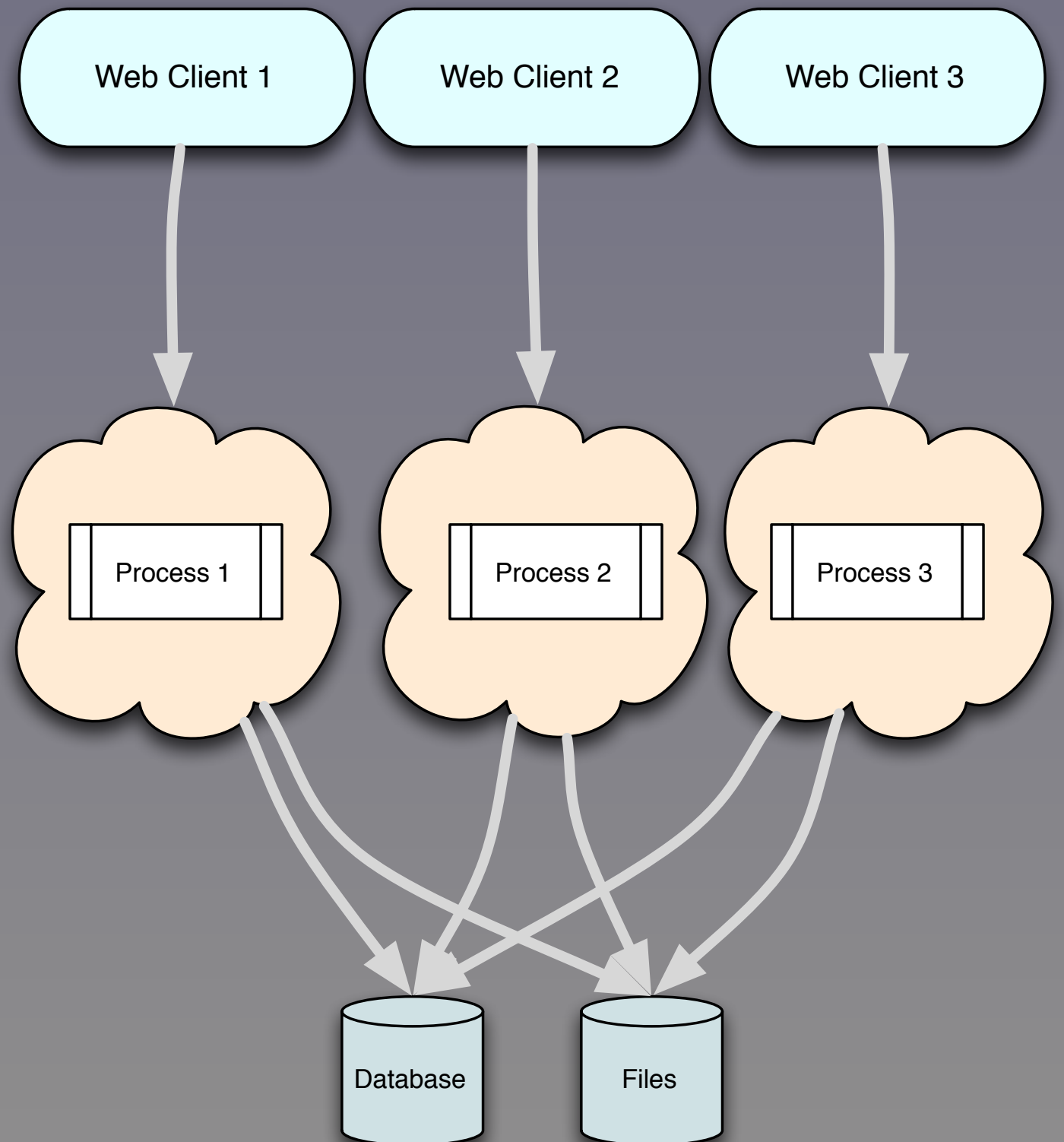
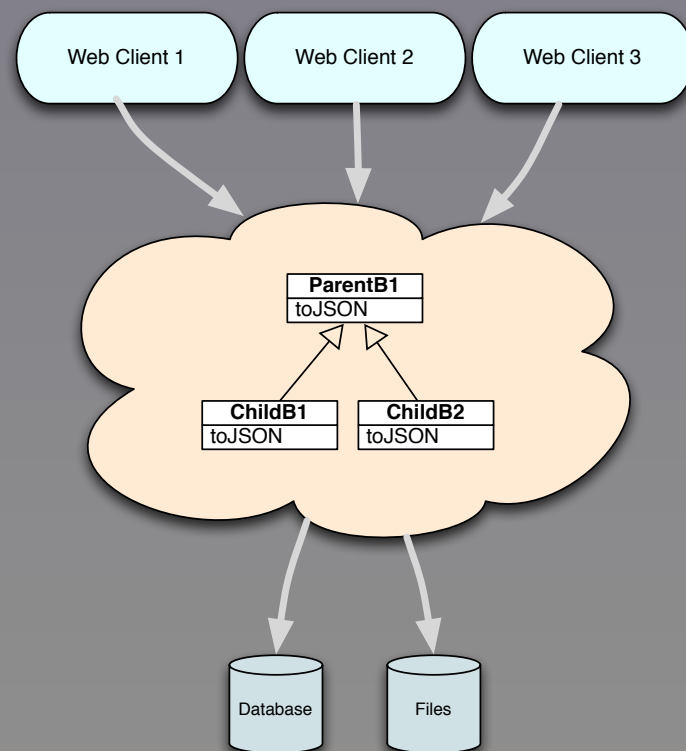
- Sets

- Trees

- ...





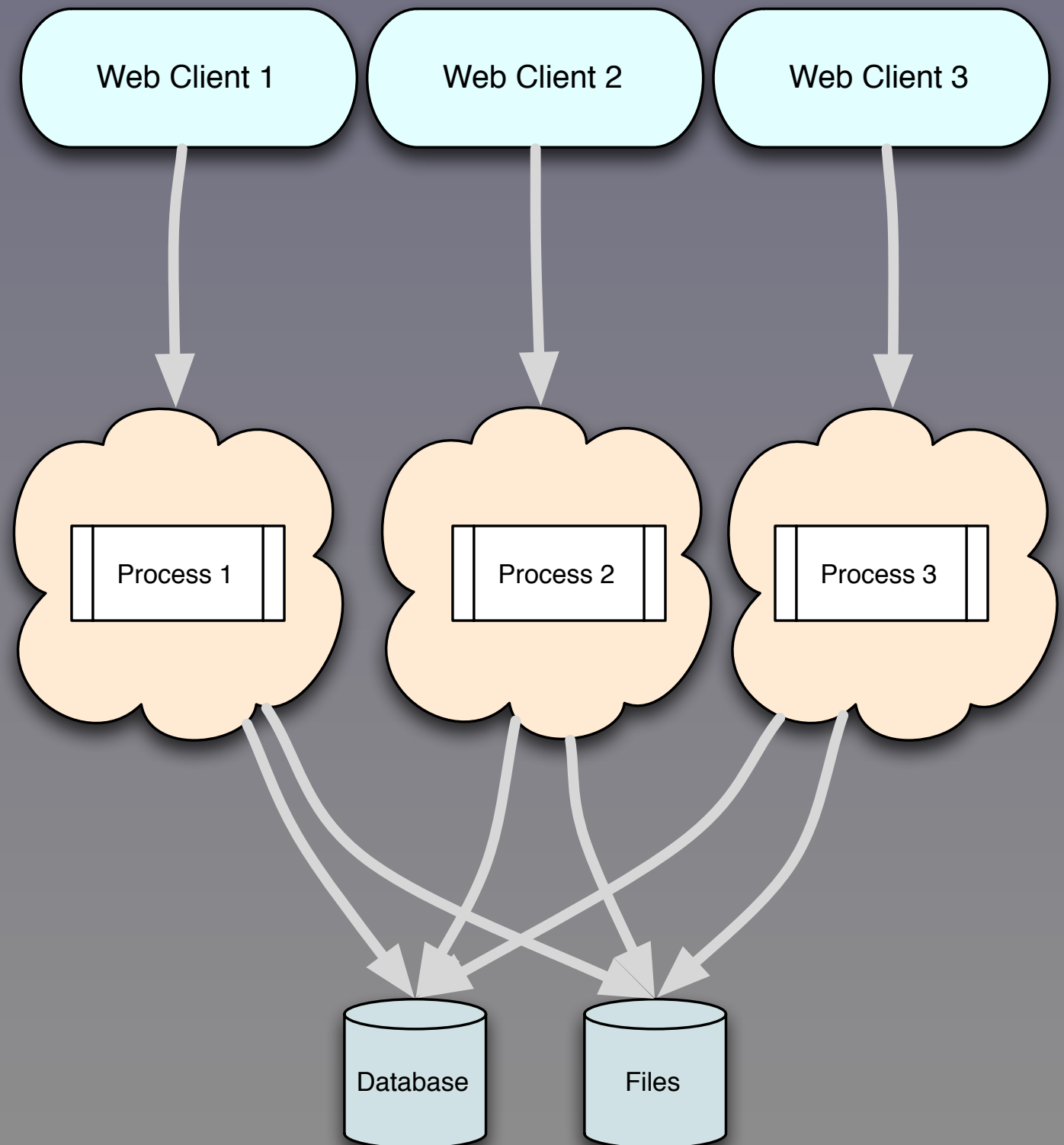


Wednesday, December 4, 13

In a broader view, object models tend to push us towards centralized, complex systems that don't decompose well and stifle reuse and optimal deployment scenarios. FP code makes it easier to write smaller, focused services that we compose and deploy as appropriate. Each "ProcessN" could be a parallel copy of another process, for horizontal, "shared-nothing" scalability, or some of these processes could be other services...

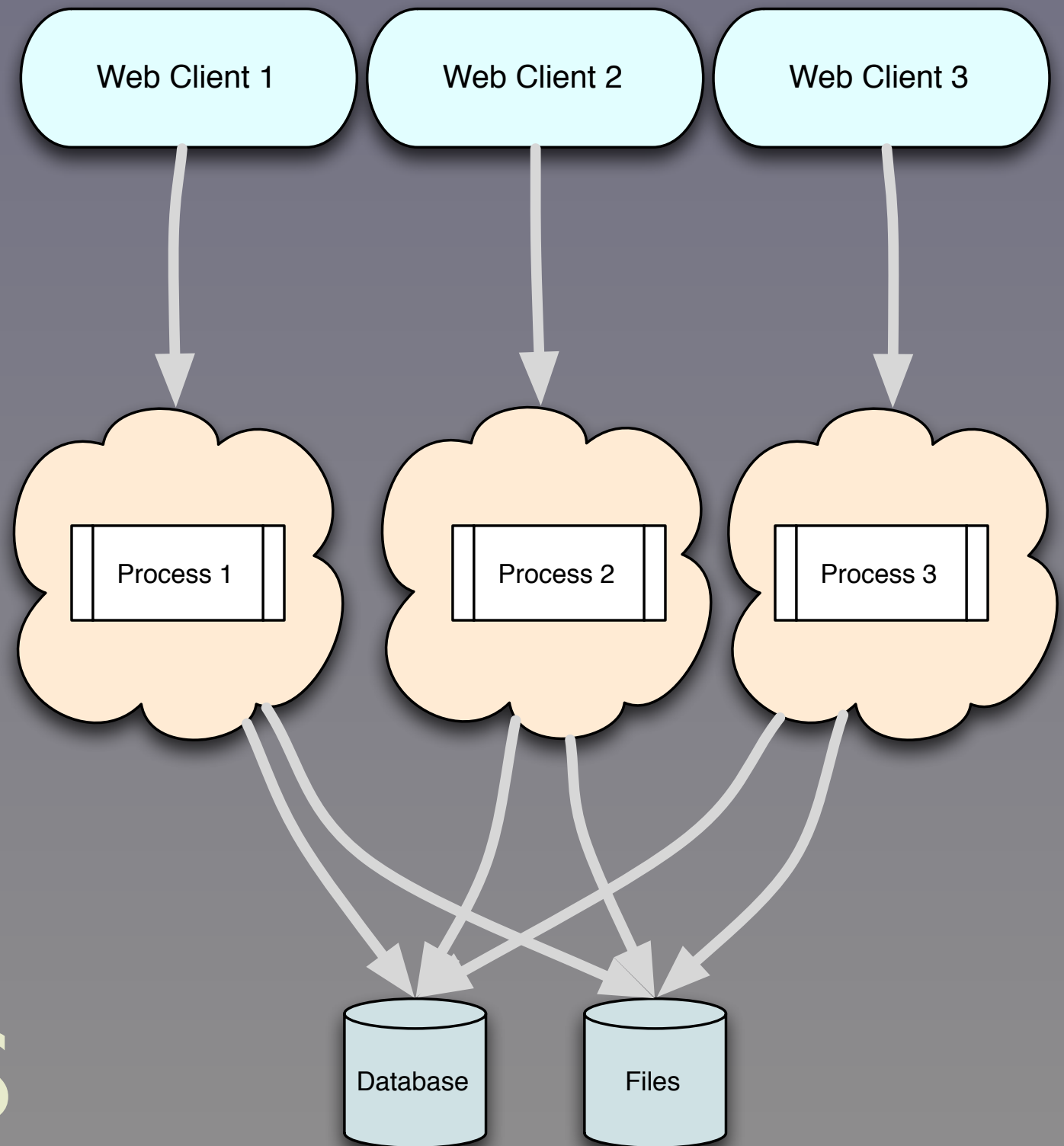
Smaller, focused services scale better, especially horizontally. They also don't encapsulate more business logic than is required, and this (informal) architecture is also suitable for scaling ML and related algorithms.

- Data Size ↑
- Formal Schema ↓
- Data-Driven Programs ↑



- MapReduce

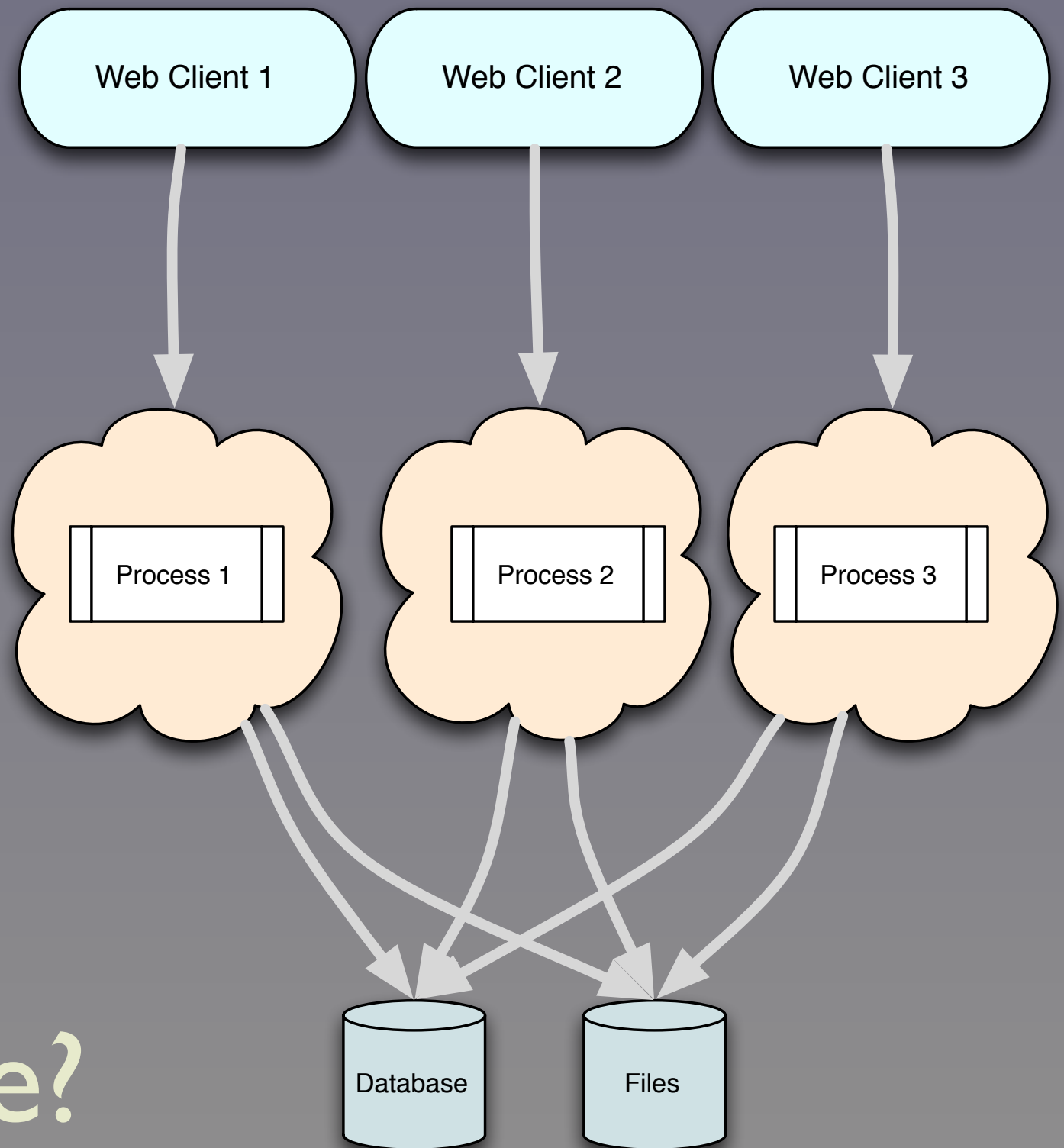
- Distributed FS



- JSON

- Node.js?

- JSON database?



What Is MapReduce?

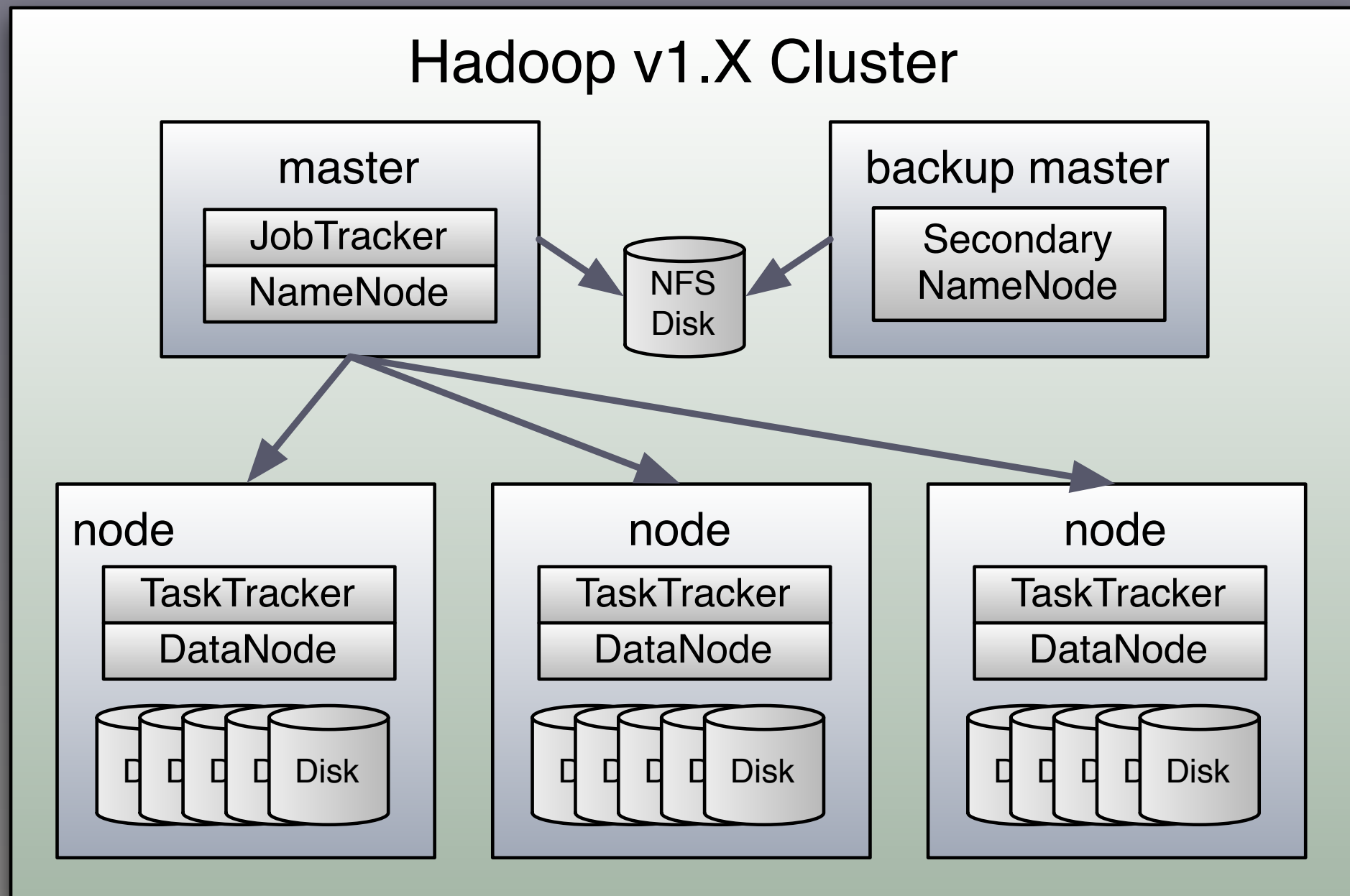


Wednesday, December 4, 13

Cloud Gate – “The Bean” – in Millenium Park, Chicago, on a sunny day – with some of my relatives ;)

Hadoop is the dominant
Big Data platform
today.

A Hadoop Cluster



Wednesday, December 4, 13

A Hadoop v1.X cluster. (V2.X introduces changes in the master processes, including support for high-availability and federation...). In brief:
JobTracker (JT): Master of submitted MapReduce jobs. Decomposes job into tasks (each a JVM process), often run where the “blocks” of input files are located, to minimize net IO.

NameNode (NN): HDFS (Hadoop Distributed File System) master. Knows all the metadata, like block locations. Writes updates to a shared NFS disk (in V1) for use by the Secondary NameNode.

Secondary NameNode (SNN): periodically merges in-memory HDFS metadata with update log on NFS disk to form new metadata image used when booting the NN and SNN.

TaskTracker: manages each task given to it by the JT.

DataNode: manages the actual blocks it has on the node.

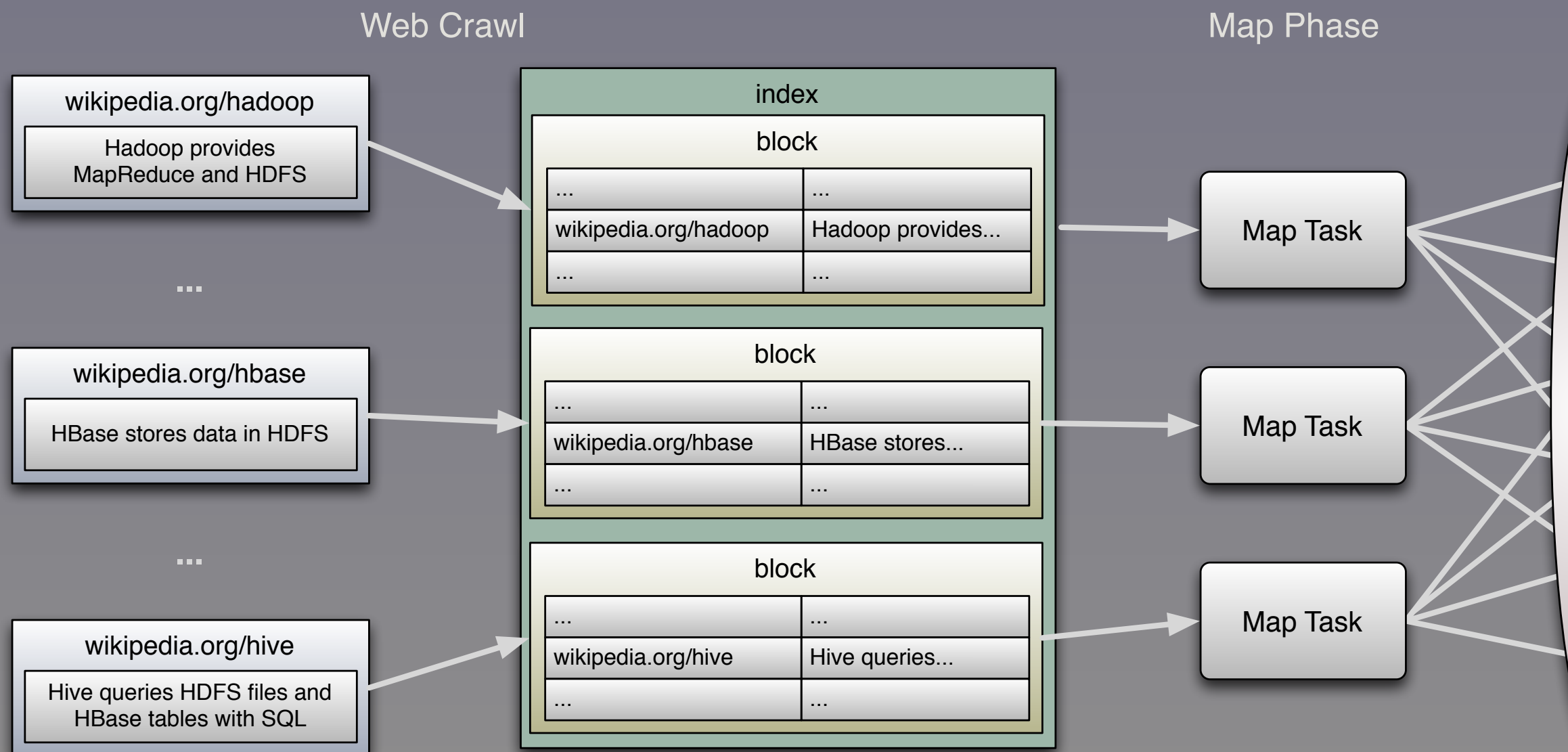
Disks: By default, Hadoop just works with “a bunch of disks” – cheaper and sometimes faster than RAID. Blocks are replicated 3x (default) so most HW failures don’t result in data loss.

MapReduce in Hadoop

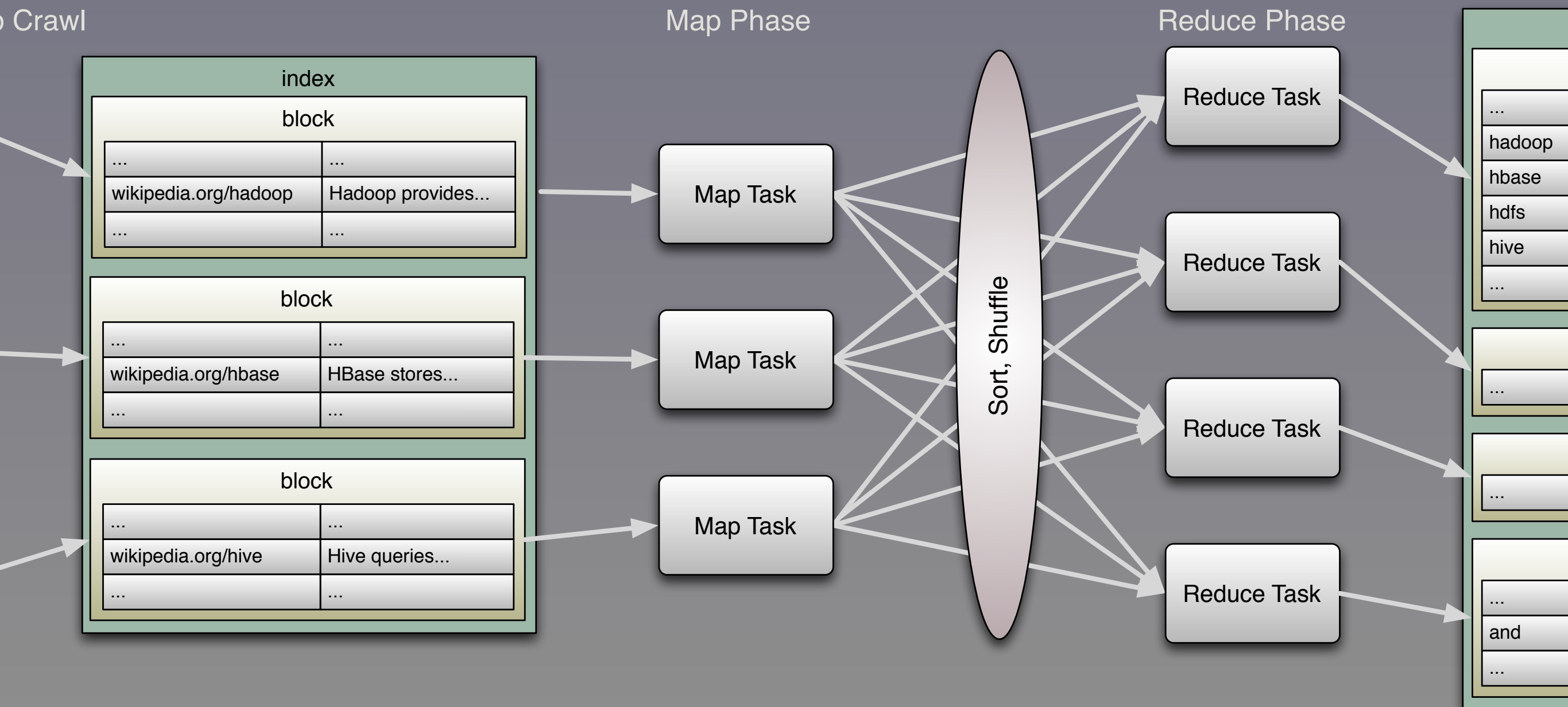
Let's look at a
MapReduce algorithm:
Inverted Index.

Used for text/web search.

Crawl teh Interwebs

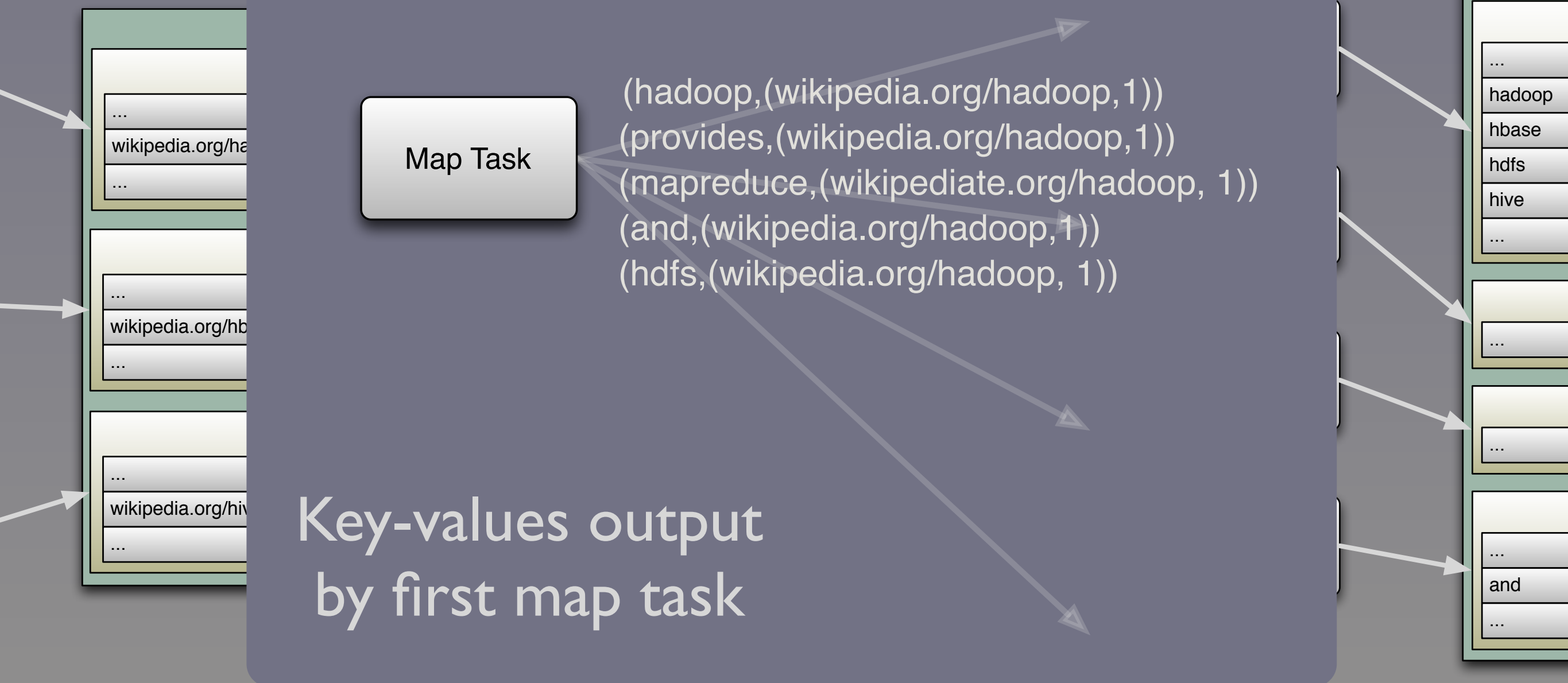


Compute Inverse Index

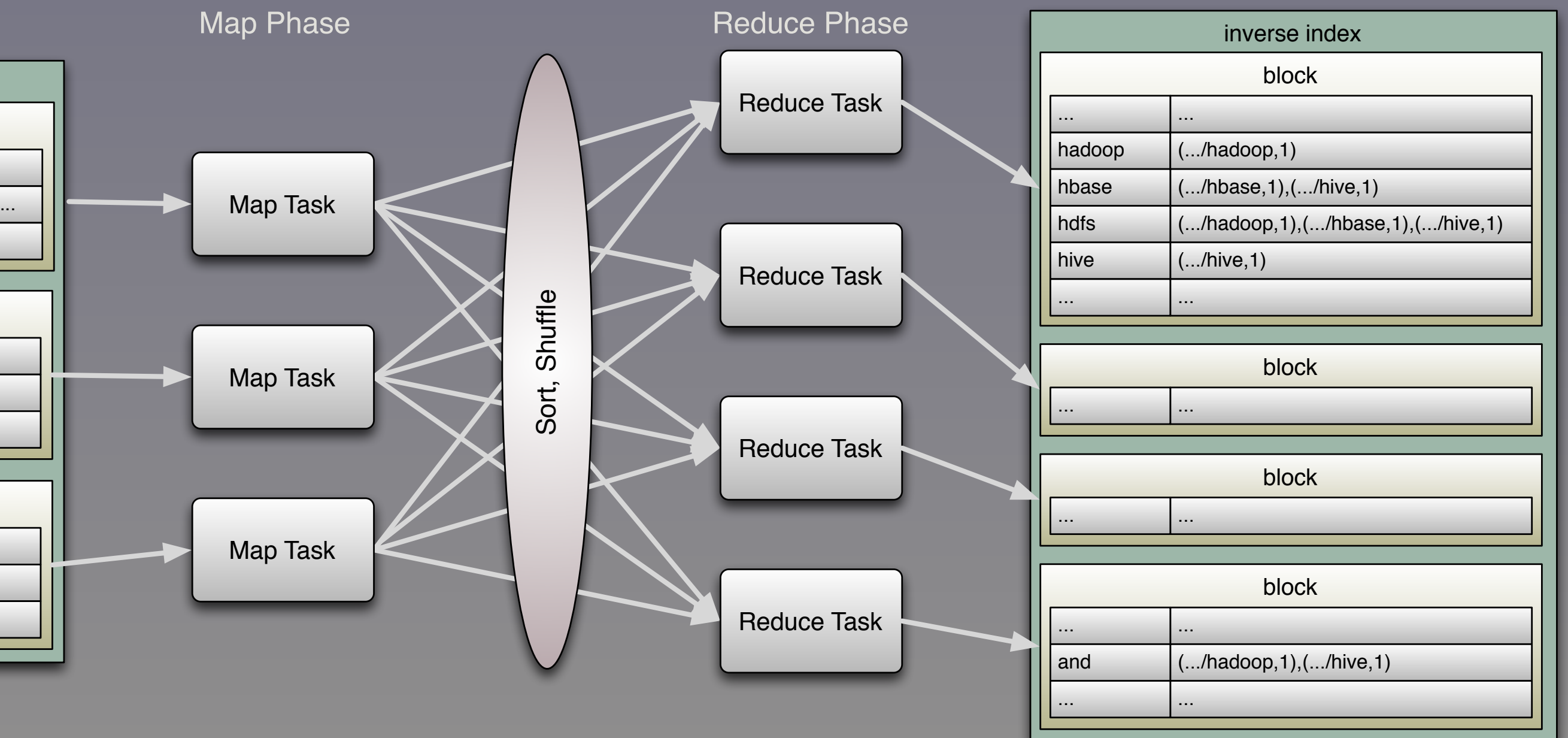


Compute Inverse Index

Crawl

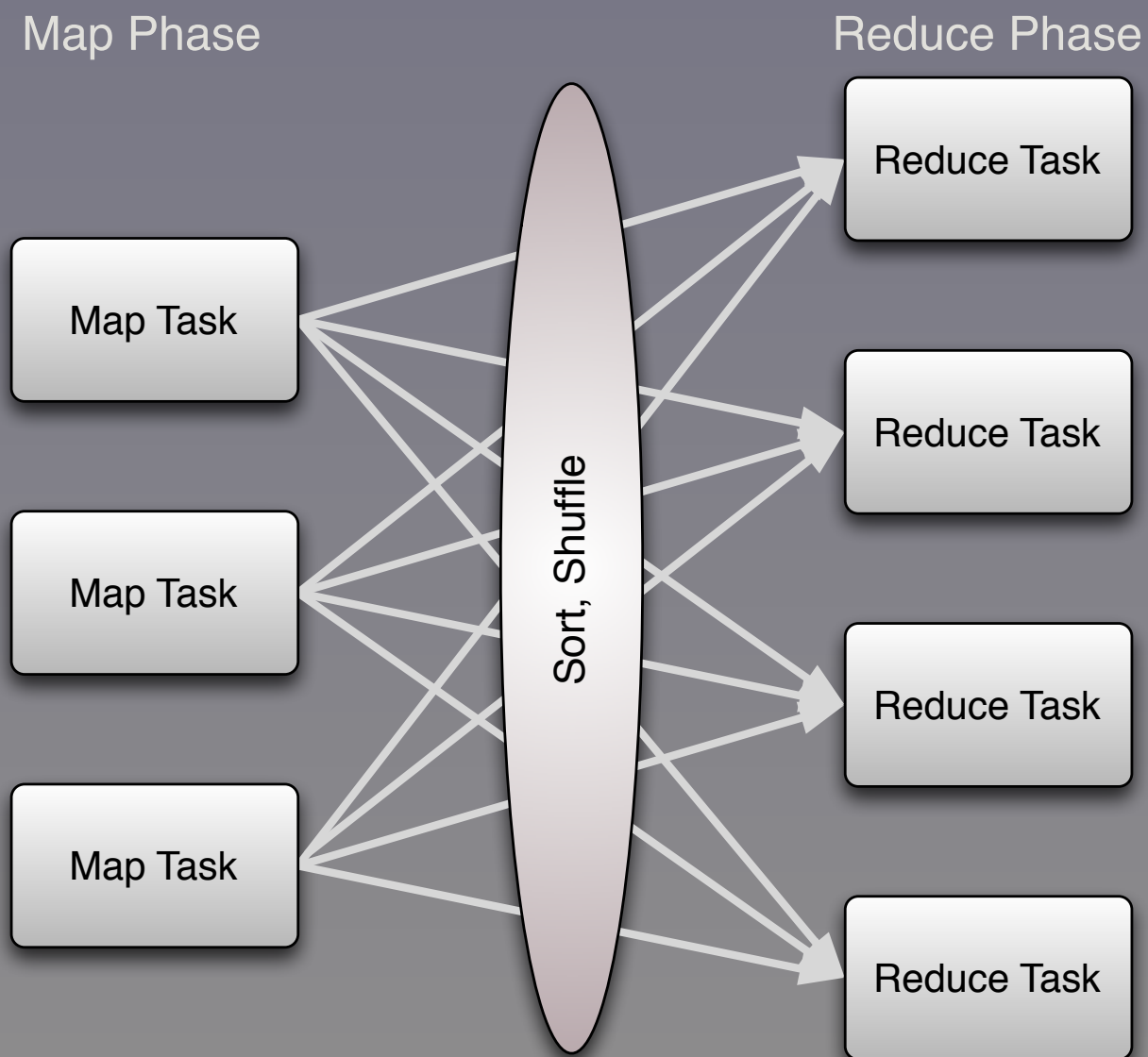


Compute Inverse Index



Finally, each reducer will get some range of the keys. There are ways to control this, but we'll just assume that the first reducer got all keys starting with "h" and the last reducer got all the "and" keys. The reducer outputs each word as a key and a list of tuples consisting of the URLs (or doc ids) and the frequency/count of the word in that document, sorted by most frequent first. (All our docs have only one occurrence of any word, so the sort is moot...)

Anatomy: MapReduce Job



Map (or Flatmap):

- Transform *one* input to *0-N* outputs.

Reduce:

- Collect *multiple* inputs into *one* output.



Andrew Whang
@whangsf



Follow

MapReduce without the Reducer
pic.twitter.com/5lQSFYmhAT

Reply Retweeted Favorite More



34
RETWEETS

16
FAVORITES



So, MapReduce is
a mashup of our friends
flatmap and reduce.

Today,
Hadoop is our best,
general-purpose tool
for horizontal scaling
of Big Data,
but...



MapReduce and Its Discontents

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Is MapReduce the end of the story? Does it meet all our needs? Let's look at a few problems...
Photo: Gratuitous Romantic beach scene, Ohio St. Beach, Chicago, Feb. 2011.

MapReduce doesn't fit
all computation needs.
HDFS doesn't fit all
storage needs.

It's hard to implement
many algorithms
in MapReduce.

Even word count is not “obvious”. When you get to fancier stuff like joins, group-bys, etc., the mapping from the algorithm to the implementation is not trivial at all. In fact, implementing algorithms in MR is now a specialized body of knowledge.

MapReduce is very
course-grained.

1-Map, 1-Reduce
phase...

Even word count is not “obvious”. When you get to fancier stuff like joins, group-bys, etc., the mapping from the algorithm to the implementation is not trivial at all. In fact, implementing algorithms in MR is now a specialized body of knowledge.

Multiple MR jobs
required for some
algorithms.

Each one flushes its
results to disk!

MapReduce is designed
for offline, batch-mode
analytics.

High latency; not
suitable for event
processing.

The Hadoop Java API
is hard to use.

Let's look at code for a
simpler algorithm,
Word Count.

(Tokenize as before, but
ignore original
document locations.)

In Word Count, the mapper just outputs the word-count pairs. We forget about the document URL/id. The reducer gets all word-count pairs for a word from all mappers and outputs each word with its final, global count.


```

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
import java.util.StringTokenizer;

class WCMapper extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

    static final IntWritable one = new IntWritable(1);
    static final Text word = new Text();    // Value will be set in a non-thread-safe way!

    @Override
    public void map(LongWritable key, Text valueDocContents,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        String[] tokens = valueDocContents.toString.split("\\s+");
        for (String wordString: tokens) {
            if (wordString.length > 0) {
                word.set(wordString.toLowerCase());
                output.collect(word, one);
            }
        }
    }
}

class Reduce extends MapReduceBase
    implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text keyWord, java.util.Iterator<IntWritable> valuesCounts,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        int totalCount = 0;
        while (valuesCounts.hasNext()) {
            totalCount += valuesCounts.next.get();
        }
        output.collect(keyWord, new IntWritable(totalCount));
    }
}

```

```

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
import java.util.StringTokenizer;

class WCMapper extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

    static final IntWritable one = new IntWritable(1);
    static final Text word = new Text();    // Value will be set in a non-thread-safe way!

    @Override
    public void map(LongWritable key, Text valueDocContents,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        String[] tokens = valueDocContents.toString.split("\\s+");
        for (String wordString: tokens) {
            if (wordString.length > 0) {
                word.set(wordString.toLowerCase());
                output.collect(word, one);
            }
        }
    }
}

class Reduce extends MapReduceBase
    implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text keyWord, java.util.Iterator<IntWritable> valuesCounts,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        int totalCount = 0;
        while (valuesCounts.hasNext()) {
            totalCount += valuesCounts.next.get();
        }
        output.collect(keyWord, new IntWritable(totalCount));
    }
}

```

The interesting bits

This is intentionally too small to read and we're not showing the main routine, which doubles the code size. The algorithm is simple, but the framework is in your face. In the next several slides, notice which colors dominate. In this slide, it's dominated by green for types (classes), with relatively few yellow functions that implement actual operations (i.e., do actual work).

The main routine I've omitted contains boilerplate details for configuring and running the job. This is just the "core" MapReduce code. In fact, Word Count is not too bad, but when you get to more complex algorithms, even conceptually simple ideas like relational-style joins and group-bys, the corresponding MapReduce code in this API gets complex and tedious very fast!


```

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
import java.util.StringTokenizer;

class WCMapper extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

    static final IntWritable one = new IntWritable(1);
    static final Text word = new Text(); // Value will be set in a non-thread-safe way!

    @Override
    public void map(LongWritable key, Text valueDocContents,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        String[] tokens = valueDocContents.toString.split("\\s+");
        for (String wordString: tokens) {
            if (wordString.length > 0) {
                word.set(wordString.toLowerCase());
                output.collect(word, one);
            }
        }
    }
}

class Reduce extends MapReduceBase
    implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text keyWord, java.util.Iterator<IntWritable> valuesCounts,
        OutputCollector<Text, IntWritable> output, Reporter reporter) {
        int totalCount = 0;
        while (valuesCounts.hasNext()) {
            totalCount += valuesCounts.next.get();
        }
        output.collect(keyWord, new IntWritable(totalCount));
    }
}

```

2000 called. It wants
its EJBs back!



Use Cascading (Java)

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Cascading is a Java library that provides higher-level abstractions for building data processing pipelines with concepts familiar from SQL such as a joins, group-bys, etc. It works on top of Hadoop's MapReduce and hides most of the boilerplate from you.

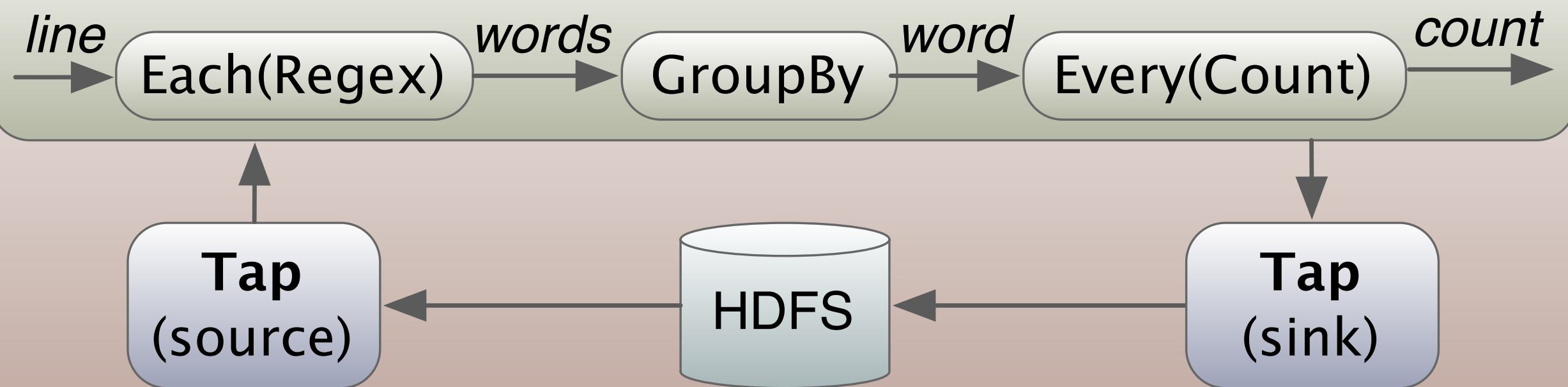
See <http://cascading.org>.

Photo: Fermi Lab Office Building, Batavia, Illinois, USA (Fermi Lab is a large particle physics accelerator facility.)

Word Count: Cascading

Flow

Pipe ("word count assembly")




```

import org.cascading.*;
...
public class WordCount {
    public static void main(String[] args) {
        String inputPath  = args[0];
        String outputPath = args[1];
        Properties properties = new Properties();
        FlowConnector.setApplicationJarClass( properties, Main.class );

        Scheme sourceScheme = new TextLine( new Fields( "line" ) );
        Scheme sinkScheme = new TextLine( new Fields( "word", "count" ) );
        Tap source = new Hfs( sourceScheme, inputPath );
        Tap sink    = new Hfs( sinkScheme, outputPath, SinkMode.REPLACE );

        Pipe assembly = new Pipe( "wordcount" );

        String regex = "(?<!\pL)(?=\pL)[^ ]*(?<=\pL)(?!\\pL)";
        Function function = new RegexGenerator( new Fields( "word" ), regex );
        assembly = new Each( assembly, new Fields( "line" ), function );
        assembly = new GroupBy( assembly, new Fields( "word" ) );
        Aggregator count = new Count( new Fields( "count" ) );
        assembly = new Every( assembly, count );

        FlowConnector flowConnector = new FlowConnector( properties );
        Flow flow = flowConnector.connect( "word-count", source, sink, assembly);
        flow.complete();
    }
}

```

Wednesday, December 4, 13

Here is the Cascading Java code. It's cleaner than the MapReduce API, because the code is more focused on the algorithm with less boilerplate, although it looks like it's not that much shorter. HOWEVER, this is all the code, where as previously I omitted the setup (main) code. See <http://docs.cascading.org/cascading/1.2/userguide/html/ch02.html> for details of the API features used here; we won't discuss them here, but just mention some highlights.

Note that there is still a lot of green for types, but at least the API emphasizes composing behaviors together.



Use Scalding (Scala)

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Scalding is a Scala “DSL” (domain-specific language) that wraps Cascading providing an even more intuitive and more boilerplate-free API for writing MapReduce jobs. <https://github.com/twitter/scalding>

Scala is a new JVM language that modernizes Java’s object-oriented (OO) features and adds support for functional programming, as we discussed previously and we’ll revisit shortly.


```

import com.twitter.scalding._

class WordCountJob(args: Args)
  extends Job(args) {
  TextLine( args("input") )
    .read
    .flatMap('line -> 'word) {
      line: String =>
        line.trim.toLowerCase
          .split("\\W+")
    }
    .groupBy('word) {
      group => group.size('count)
    }
  }
  .write(Tsv(args("output")))
}

```

That's It!!



Use Cascalog (Clojure)

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

<http://nathanmarz.com/blog/introducing-cascalog-a-clojure-based-query-language-for-hado.html>

Clojure is a new JVM, lisp-based language with lots of important concepts, such as persistent datastructures.


```
(defn lowercase [w] (.toLowerCase w))
```

```
(?<- (stdout) [?word ?count]  
  (sentence ?s)  
    (split ?s :> ?word1)  
  (lowercase ?word1 :> ?word)  
    (c/count ?count))
```

Datalog-style queries



Moving Beyond MapReduce Use Spark

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

<http://www.spark-project.org/>

Why isn't it more widely used? 1) lack of commercial support, 2) only recently emerged out of academia.


```
object WordCountSpark {  
  def main(args: Array[String]) {  
    val file = spark.textFile(args(0))  
    val counts = file.flatMap(  
      line => line.split("\\W+"))  
      .map(word => (word, 1))  
      .reduceByKey(_ + _)  
    counts.saveAsTextFile(args(1))  
  }  
}
```

Also small and concise!

Spark is a Hadoop MapReduce alternative:

- Distributed computing with in-memory caching.
- ~10-100x faster than MapReduce (in part due to caching of intermediate data).

Spark is a Hadoop MapReduce alternative:

- Originally designed for machine learning applications.
- Developed by Berkeley AMP.



Use SQL! Hive, Shark, Impala, Presto, or Lingual

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Using SQL when you can! Here are 5 (and growing!) options, some of which still use MapReduce, while others have introduced new, faster runtimes.

Here, we're discussing SQL as a tool for computation and not discussing the storage aspect of SQL database systems.

Use SQL when you can!

- Hive: SQL on top of MapReduce.
- Shark: Hive ported to Spark.
- Impala & Presto: HiveQL with new, faster back ends.
- Lingual: ANSI SQL on Cascading.

Word Count in Hive SQL!

```
CREATE TABLE docs (line STRING);  
LOAD DATA INPATH '/path/to/docs'  
INTO TABLE docs;
```

```
CREATE TABLE word_counts AS  
SELECT word, count(1) AS count FROM  
(SELECT explode(split(line, '\W+'))  
  AS word FROM docs) w  
GROUP BY word  
ORDER BY word;
```

Works for Hive, Shark, and Impala

We're in the era I call
The SQL Strikes Back!

(with apologies to
George Lucas...)

IT shops realize that NoSQL is useful and all, but people really, Really, REALLY love SQL. So, it's making a big comeback. You can see it in Hadoop, in SQL-like APIs for some “NoSQL” DBs, e.g., Cassandra and MongoDB's Javascript-based query language, as well as “NewSQL” DBs.

Hadoop owes a lot of its popularity to Hive!

Some “NoSQL”
databases have or are
adding query languages
(e.g., Cassandra,
MongoDB).

“NewSQL” databases
are bringing NoSQL
performance to the
relational model.

Examples

- Google Spanner and F1.
- NuoDB.
- VoltDB.

Spanner is the successor to BigTable. It is a globally-distributed database (consistency is maintained using the Paxos algo. and hardware synchronized clocks through GPS and atomic clocks!) Each table requires a primary key. F1 is an RDBMS built on top of it.

NuoDB is a cloud based RDBMS.

VoltDB is an example “in-memory” database, which are ideal for lots of small transactions that leverage indexing and rarely require full table scans.

(So, take what I said earlier:
Formal Schemas ↓
with a grain of salt...)

MapReduce is not
suitable for
event processing
("real-time").



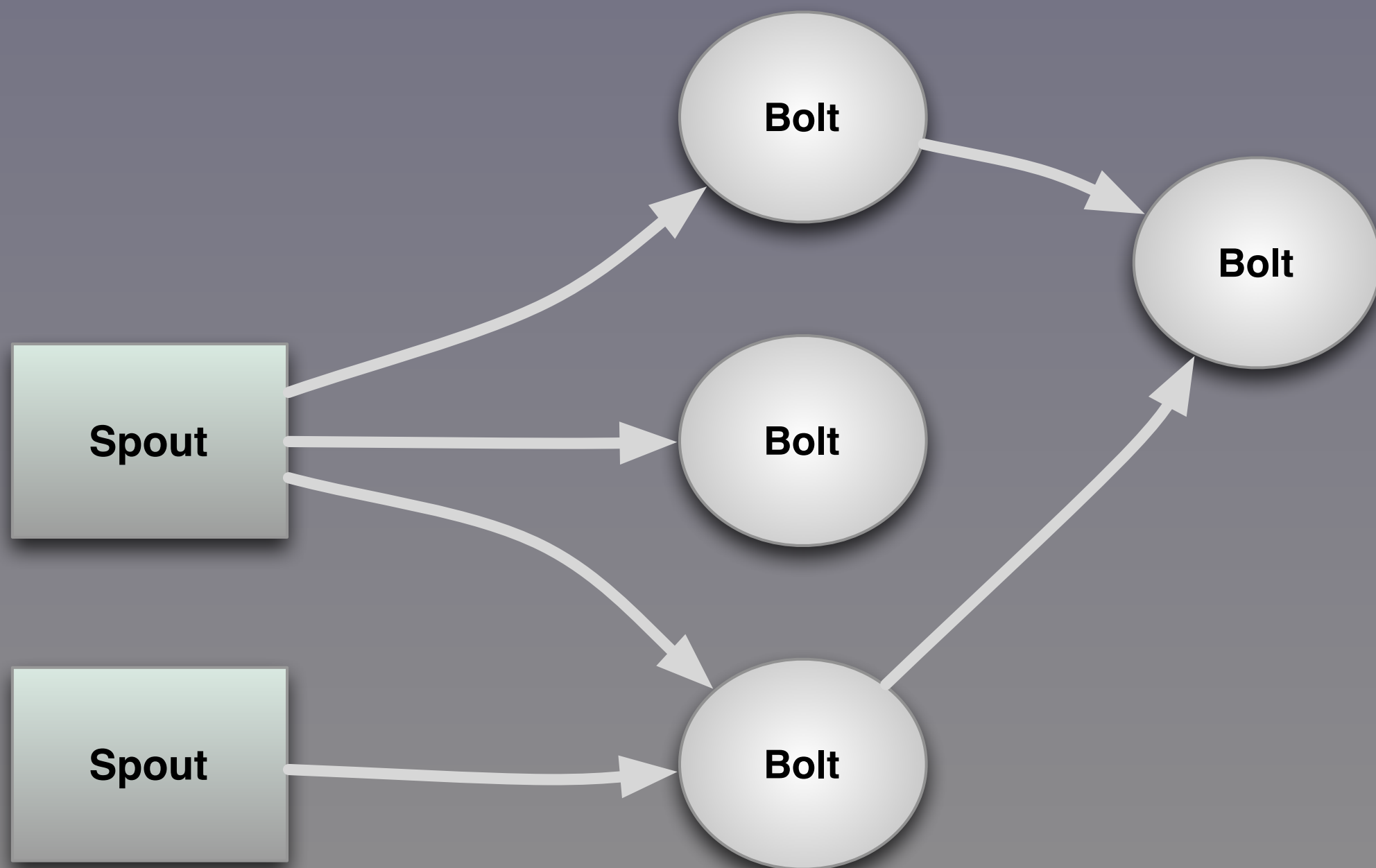
Storm!

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Photo: Top of the AON Building, Chicago, after a Storm passed through.

Storm implements
reliable, distributed
event processing.



In Storm terminology, Spouts are data sources and bolts are the event processors. There are facilities to support reliable message handling, various sources encapsulated in Sprouts and various targets of output. Distributed processing is baked in from the start.

Spark and Message
Queues are also being
used for distributed
event processing.

Databases to the Rescue?



Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Databases as a real-time event processing option?

Photo: Outside my condo window, Chicago!

SQL or NoSQL Databases?

Capture events in a database
with fast writes.

HDFS?

HDFS is oriented towards batch-mode reads and writes.

So, it's not suitable for incremental updates, like capturing events.

Hadoop vs. SQL?

- Hadoop
- Very flexible compute model
- “Table” scans
- Batch mode
- NoSQL / SQL
- Focused on a data model
- Transactional
- Event driven

Problem:

Your current data
warehouse can only
store 6-months of data
without a \$1M upgrade.

Traditional DW

- *Pros*
 - Mature
 - Rich SQL, analytics
 - Mid-size data
- *Cons*
 - Expensive - \$/TB
 - Scalability limits

SQL is very important
for data warehouse
applications,
but transactions aren't.

NoSQL does give you the more cost-effective storage, but SQL is very important for most DW applications, so your “NoSQL” store would need a powerful query tool to support common DW scenarios. However, DW experts usually won't tolerate anything that isn't SQL. Note that Cassandra is one of several NoSQL and “NewSQL” databases with a SQL dialect.

- Traditional DW
 - + Mature
 - + Rich SQL, analytics
 - Scalability
 - \$\$/TB
- Hadoop
 - Less mature
 - + Improving SQL
 - + Scalable!
 - + Low \$/TB

Data warehouses tend to be more scalable and a little less expensive than OLTP systems, which is why they are used to “warehouse” transactional data and perform analytics. However, their \$/TB is ~10x the cost on Hadoop and Hadoop scales to larger data sets.

Hadoop has become a
popular data warehouse
supplement/replacement.

Many of my projects have offloaded an overburdened or expensive traditional data warehouse to Hadoop. Sometimes a wholesale replacement, but more often a supplemental strategy, at least for a transitional period of some duration.

MapReduce is not ideal
for graph algorithms.

Graph Systems



Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

A good summary presentation: <http://www.slideshare.net/shatteredNirvana/pregel-a-system-for-largescale-graph-processing>

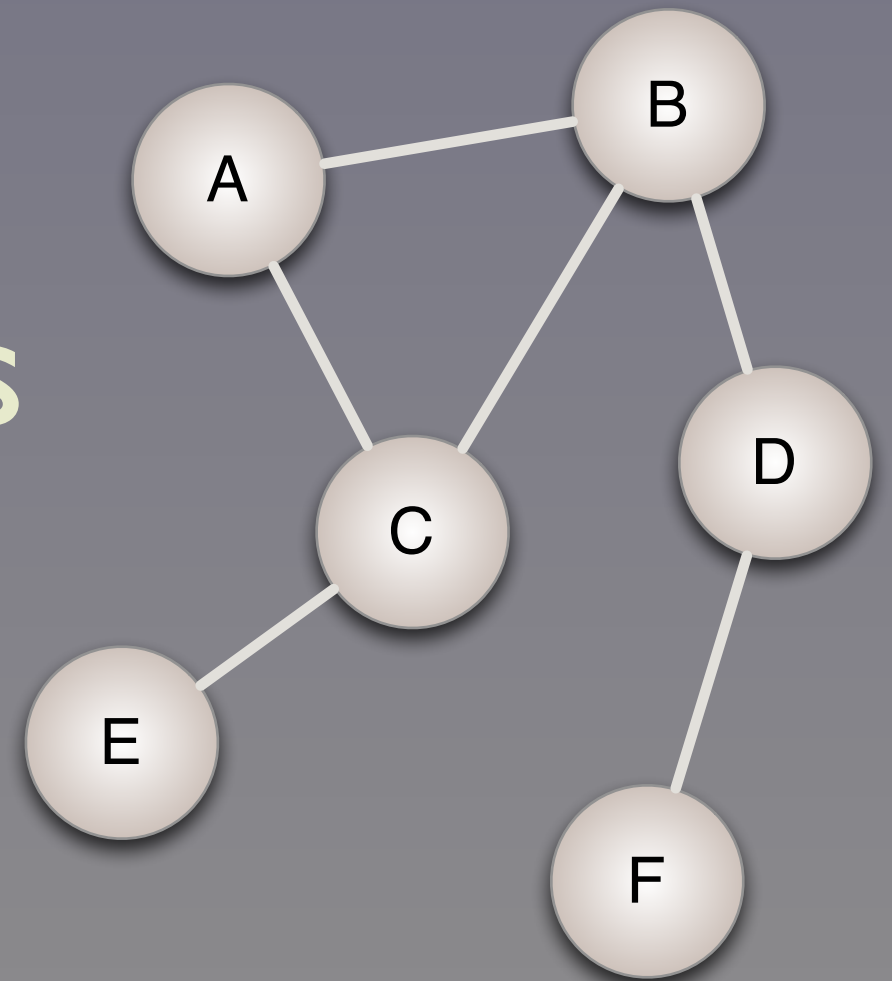
Photo: Detail of the now-closed Esquire Movie Theater, a few blocks from here, Feb. 2011

Google's Page Rank

Google invented MapReduce,
... but MapReduce is not ideal for
Page Rank and other graph
algorithms.

Why not MapReduce?

- 1 MR job for each iteration that updates all n nodes/edges.
- Graph saved to disk after each iteration.
- ...



Google's Pregel

- Pregel: New graph framework for Page Rank.
- Bulk, Synchronous Parallel (BSP).
- Graphs are first-class citizens.
- Efficiently processes updates...

Open-source Alternatives

- Apache Giraph.
- Apache Hama.
- Aurelius Titan.

All are
somewhat
immature.

Sometimes, you need a
specialized tool.

Purpose-built Tools



Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

I see that a trend where completely generic tooling is giving way to more “purpose-built” tooling...

Photo: Buildings along the Chicago River.

New compute engines
(e.g., Impala, Presto).

New Hadoop
file formats, optimize
access (e.g., Parquet).

Lucene with Solr and Elasticsearch

Example of solving a
specific problem with a
custom solution.

MapReduce is not
iterative, so Machine
Learning algorithms
perform poorly.



Machine Learning

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

ML includes recommendation engines (e.g., the way Netflix recommends movies to you or Amazon recommends products), classification (e.g., SPAM classifiers, character and image recognition), and clustering. Other specialized examples include text mining and other forms of natural language processing (NLP).

Photo: Two famous 19th Century Buildings in Chicago.

- Recommendations: Netflix movies, Amazon products, ...
- Classification: SPAM filters, character recognition, ...
- Clustering: Find groups in social networks, ...

ML algorithms
tend to be iterative,
but they can be force fit
into MapReduce.

Machine Learning Tools

- Mahout: MapReduce algorithms.
- Pattern: PMML on Cascading.
- Spark: More flexible compute model.

Common Workflow

- Train ML models with Hadoop.
- Store model in a database.
- Predict based on user events.

Emerging: Probabilistic Programming

- Languages for Probabilistic Graphical Models??
- Bayesian Networks.
- Markov Chains.
- ...

So, where are we??



Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

Now that we have cataloged some issues and solutions, let's recap and look forward.
Photo: Lake Michigan, near Ohio Street Beach, Chicago, Feb. 2011.



Hadoop MapReduce is the Enterprise Java Beans of our time.

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

I worked with EJBs a decade ago. The framework was completely invasive into your business logic. There were too many configuration options in XML files. The framework “paradigm” was a poor fit for most problems (like soft real time systems and most algorithms beyond Word Count). Internally, EJB implementations were inefficient and hard to optimize, because they relied on poorly considered object boundaries that muddled more natural boundaries. (I’ve argued in other presentations and my “FP for Java Devs” book that OOP is a poor modularity tool...) The fact is, Hadoop reminds me of EJBs in almost every way. It’s a 1st generation solution that mostly works okay and people do get work done with it, but just as the Spring Framework brought an essential rethinking to Enterprise Java, I think there is an essential rethink that needs to happen in Big Data, specifically around Hadoop. The functional programming community, is well positioned to create it...

A photograph of a city skyline at dusk. A large, dark plume of smoke or steam rises from the horizon, spreading across the sky. An airplane is visible in the upper left portion of the sky. The text "MapReduce is waning" is overlaid on the right side of the image.

MapReduce is waning

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

We've seen a lot of issues with MapReduce. Already, alternatives are being developed, either general options, like Spark and Storm, or special-purpose built replacements, like Impala. Let's consider other options...

Emerging replacements are based on Functional Languages...

```
import com.twitter.scalding._

class WordCountJob(args: Args) extends Job(args) {
  TextLine( args("input") )
    .read
    .flatMap('line -> 'word) {
      line: String =>
        line.trim.toLowerCase
          .split("\\W+")
    }
    .groupBy('word) {
      group => group.size('count) }
    .write(Tsv(args("output")))
}
```

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

FP is such a natural fit for the problem that any attempts to build big data systems without it will be handicapped and probably fail.

Let's consider other MapReduce options...

... and SQL,
which is roaring back!

```
CREATE TABLE docs (line STRING);  
LOAD DATA INPATH '/path/to/docs'  
INTO TABLE docs;
```

```
CREATE TABLE word_counts AS  
SELECT word, count(1) AS count FROM  
(SELECT explode(split(line, '\W+'))  
  AS word FROM docs) w  
GROUP BY word  
ORDER BY word;
```

Copyright © 2011-2013, Dean Wampler, All Rights Reserved

Wednesday, December 4, 13

FP is such a natural fit for the problem that any attempts to build big data systems without it will be handicapped and probably fail.

Let's consider other MapReduce options...

Why are Scala, Clojure,
and SQL solutions so
concise and appealing?

Big Data is
Mathematics.
∴ Functional Languages
are the best tools.

Concurrency
has been called
the killer app for FP.

Big Data is a
bigger killer app, IMHO.

Questions?

CodeMesh 2013, December 4
dean.wampler@typesafe.com
[@deanwampler](https://twitter.com/deanwampler)
polyglotprogramming.com/talks

Copyright © 2011-2013, Dean Wampler, All Rights Reserved



Wednesday, December 4, 13

All pictures Copyright © Dean Wampler, 2011–2013, All Rights Reserved. All other content is free to use, but attribution is requested.

Photo: Building in fog on Michigan Avenue, Chicago.